

Отчёт по лабораторной работе №4

Выполнил студент НКАбд-02-25

Валерия Сергеевна Григорьева

Содержание

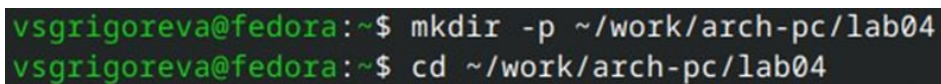
1	Цель работы	3
2	Порядок выполнения лабораторной работы	4
3	Задание для самостоятельной работы	8
4	Вывод	10

1 Цель работы

Цель данной работы — освоение процедуры компиляции и сборки программ, написанных на ассемблере NASM.

2 Порядок выполнения лабораторной работы

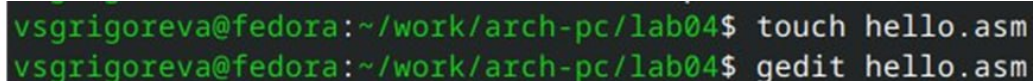
В начале создала каталог для работы с программами на языке ассемблера NASM, а затем перешла в него(Рисунок 2.1).



```
vsgrigoreva@fedora:~$ mkdir -p ~/work/arch-pc/lab04
vsgrigoreva@fedora:~$ cd ~/work/arch-pc/lab04
```

Рисунок 2.1: Создание каталога

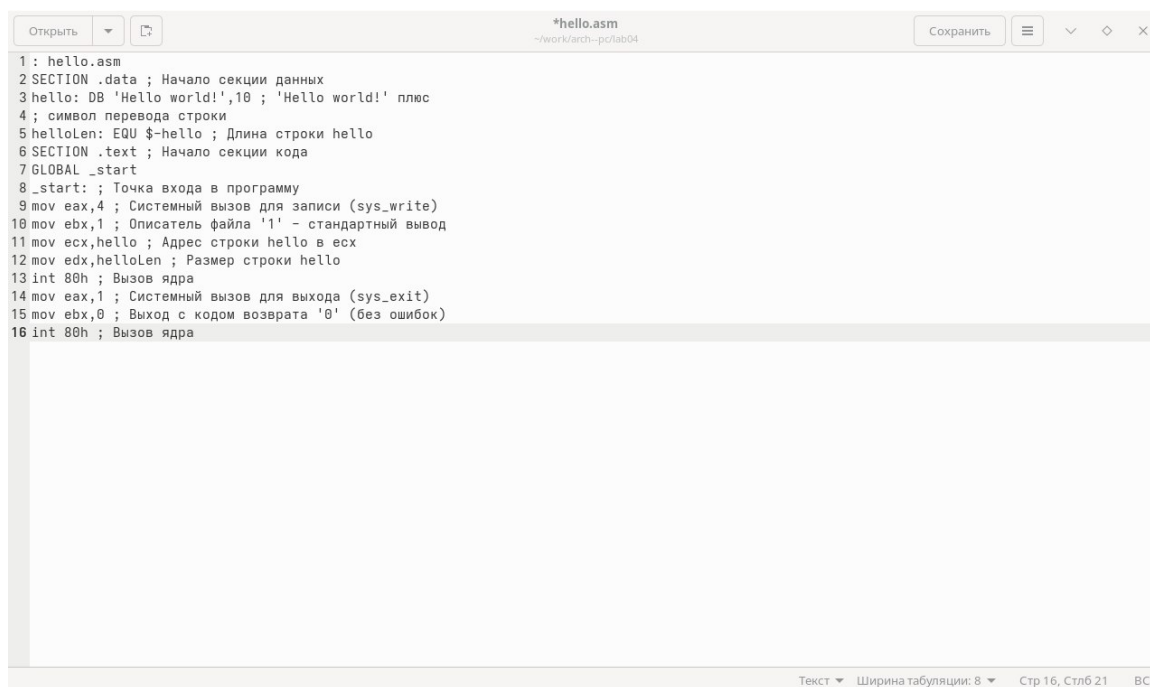
Далее создала текстовый файл с именем hello.asm с помощью команды touch и открыла его с помощью gedit(Рисунок 2.2).



```
vsgrigoreva@fedora:~/work/arch-pc/lab04$ touch hello.asm
vsgrigoreva@fedora:~/work/arch-pc/lab04$ gedit hello.asm
```

Рисунок 2.2: Создание файла и его открытие

Текстовый файл открылся, в него необходимо было ввести текст(Рисунок 2.3).

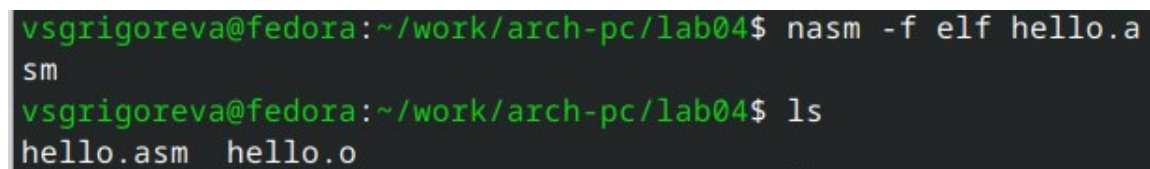


```
1: hello.asm
2 SECTION .data ; Начало секции данных
3 hello: DB 'Hello world!',10 ; 'Hello world!' плюс
4 ; символ перевода строки
5 hellolen: EQU $-hello ; Длина строки hello
6 SECTION .text ; Начало секции кода
7 GLOBAL _start
8 _start: ; Точка входа в программу
9 mov eax,4 ; Системный вызов для записи (sys_write)
10 mov ebx,1 ; Описатель файла '1' - стандартный вывод
11 mov ecx,hello ; Адрес строки hello в ecx
12 mov edx,hellolen ; Размер строки hello
13 int 80h ; Вызов ядра
14 mov eax,1 ; Системный вызов для выхода (sys_exit)
15 mov ebx,0 ; Выход с кодом возврата '0' (без ошибок)
16 int 80h ; Вызов ядра
```

Рисунок 2.3: Текст программы «Hello World»

В ассемблерной программе каждая команда располагается на отдельной строке, а синтаксис ассемблера NASM является чувствительным к регистру.

Чтобы скомпилировать приведенный выше текст программы «Hello World», я использовала транслятор NASM с помощью команды «`nasm -f elf hello.asm`». Текст программы был набран правильно, поэтому транслятор преобразовал его из файла `hello.asm` в объектный код, записанный в файл `hello.o`. Ключ `-f` указывает транслятору, что требуется создать бинарные файлы в формате ELF. Далее с помощью команды `ls` проверила, что нужный файл был создан (Рисунок 2.4). Объектный файл имеет имя `hello.o`.



```
vsgrigoreva@fedora:~/work/arch-pc/lab04$ nasm -f elf hello.a
sm
vsgrigoreva@fedora:~/work/arch-pc/lab04$ ls
hello.asm  hello.o
```

Рисунок 2.4: Превращение текста программы в объектный код и проверка создания файла

Далее я выполнила команду, которая скомпилировала исходный файл `hello.asm` в `obj.o` (опция `-o` позволяет задать имя объектного файла, в данном случае `obj.o`), при этом формат выходного файла `elf`, в него включены символы для отладки с помощью опции `-g`, кроме того был создан файл листинга `list.lst` с помощью опции `-l`. Затем удостоверилась, что файлы были созданы с помощью `ls` (Рисунок 2.5).

```
vsgrigoreva@fedora: ~/work/arch-pc/lab04$ nasm -o obj.o -f elf -g -l list.lst hello.asm
vsgrigoreva@fedora: ~/work/arch-pc/lab04$ ls
hello.asm  hello.o  list.lst  obj.o
```

Рисунок 2.5: Выполнение команды

Затем объектный файл я передала на обработку компоновщику, чтобы получить исполняемую программу. Далее я проверила, что исполняемый файл `hello` был создан (Рисунок 2.6).

```
vsgrigoreva@fedora: ~/work/arch-pc/lab04$ ld -m elf_i386 hello.o -o hello
vsgrigoreva@fedora: ~/work/arch-pc/lab04$ ls
hello  hello.asm  hello.o  list.lst  obj.o
```

Рисунок 2.6: Компоновщик LD

Компоновщик `ld` не предполагает по умолчанию расширений для файлов, но принято использовать следующие расширения: `o` – для объектных файлов; без расширения – для исполняемых файлов; `tar` – для файлов схемы программы; `lib` – для библиотек. Ключ `-o` с последующим значением задаёт в данном случае имя создаваемого исполняемого файла.

Затем я выполнила команду `ld -m elf_i386 obj.o -o main` (Рисунок 2.7).

```
vsgrigoreva@fedora: ~/work/arch-pc/lab04$ ld -m elf_i386 obj.o -o main
vsgrigoreva@fedora: ~/work/arch-pc/lab04$ ls
hello  hello.asm  hello.o  list.lst  main  obj.o
```

Рисунок 2.7: Выполнение команды

Исполняемый файл имеет имя `main`, объектный файл, из которого собран этот исполняемый файл – `obj.o`.

Чтобы запустить на выполнение созданный исполняемый файл, находящийся в текущем каталоге, я набрала в командной строке `./hello` (Рисунок 2.8).

A terminal window with a dark background. The prompt is 'vsgrigoreva@fedora:~/work/arch-pc/lab04\$' in green. The command './hello' is entered in white. The output 'Hello world!' is displayed in white on the next line.

```
vsgrigoreva@fedora:~/work/arch-pc/lab04$ ./hello
Hello world!
```

Рисунок 2.8: Запуск исполняемого файла

3 Задание для самостоятельной работы

1. В каталоге с лабораторной работой с помощью команды `cp` создала копию файла `hello.asm` с именем `lab4.asm`(Рисунок 2.9).



```
vsgrigoreva@fedora:~/work/arch-pc/lab4$ cp hello.asm lab4.asm
vsgrigoreva@fedora:~/work/arch-pc/lab4$ ls
hello  hello.asm  hello.o  lab4.asm  list.lst  main  obj.o
```

Рисунок 3.1: Создание копии файла `hello.asm`

- 2, 3. С помощью текстового редактора `gedit` внесла изменения в текст программы в новом файле `lab4.asm` так, чтобы на экран выводилась строка с моим именем и фамилией(Рисунок 2.10).

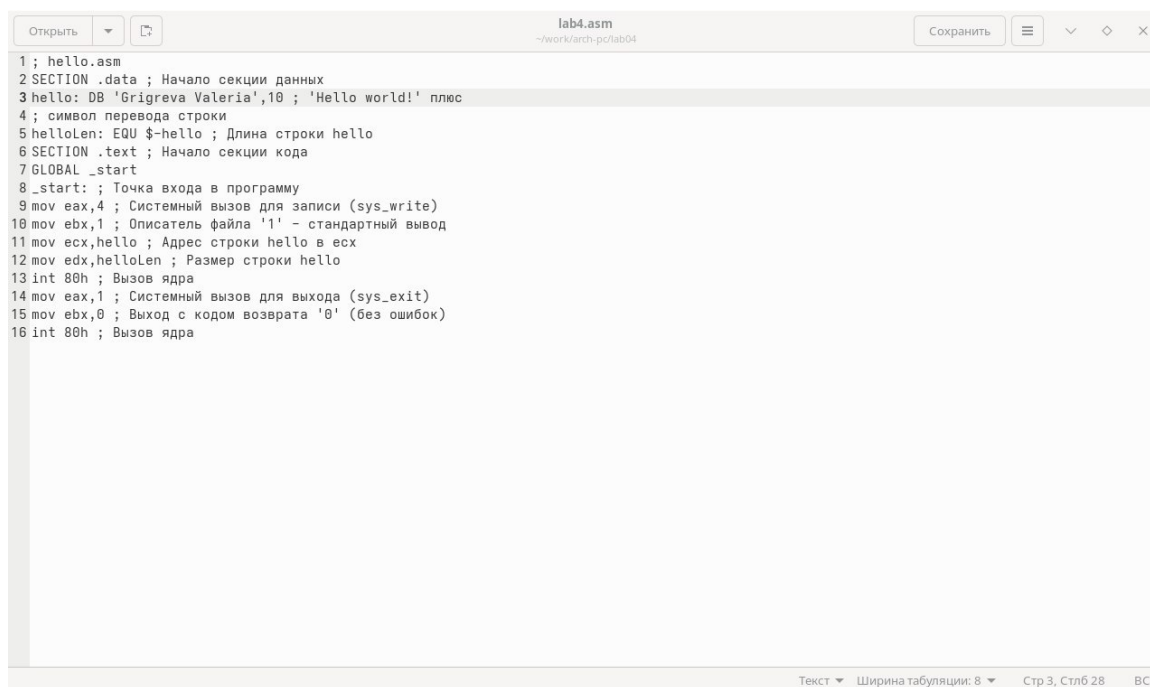


Рисунок 3.2: Изменение текста программы

Оттранслировала полученный текст в объектный файл, выполнила его компоновку и запустила получившийся исполняемый файл(Рисунок 2.11).



Рисунок 3.3: Вывод имени и фамилии

- Далее я скопировала файлы hello.asm и lab4.asm в мой локальный репозиторий и загрузила их на github.

4 Вывод

В процессе выполнения лабораторной работы я научилась создавать простую программу на языке ассемблера NASM, превращать текст программы в объектный код, компилировать файл, компоновать и запускать его.